

Step 18: Written Analysis Report

1. Executive Summary

This project analyzes the House Prices dataset using professional data visualization and feature engineering techniques. The main goal was to understand the strongest drivers of house sale price, create meaningful predictive features, encode categorical variables correctly, treat skewed numerical features, scale data, and select a final feature set for future machine learning models. The dataset contains housing attributes such as living area, quality, basement size, year built, garage capacity, neighborhood, and sale price. The top findings show that overall quality, living area, total property size, and neighborhood-related factors strongly influence sale price. Feature engineering added valuable predictors such as `TotalSF`, `TotalBaths`, `HouseAge`, and `QualCond`.

2. Visualization Insights

The visualizations provided a strong understanding of how house prices behave. The SalePrice distribution showed strong right skewness, meaning most houses are moderately priced while fewer houses have very high prices. The log transformation made the target more balanced and suitable for machine learning. The scatter plot between `GrLivArea` and `SalePrice` showed a positive relationship, especially for houses with higher overall quality. The neighborhood boxplot revealed that some neighborhoods consistently have higher median sale prices, showing location importance. The correlation heatmap identified strong predictors such as `OverallQual`, `GrLivArea`, `GarageCars`, and engineered area features. The most informative visualization was the final dashboard because it combined distribution, relationship, feature importance, quality comparison, correlation, and scaling views in one place.

3. Feature Engineering Rationale

Eight engineered features were created using domain knowledge. `TotalSF` combines basement, first-floor, and second-floor areas to represent total usable property space. `TotalBaths` creates a weighted bathroom count because full bathrooms usually add more value than half bathrooms. `HouseAge` captures the age of the house at the time of sale, while `RemodelAge` measures how recently the property was remodeled. `HasRemodeled` identifies whether a house has been updated after construction. `QualCond` captures the interaction between overall quality and overall condition. `PricePerSF` measures price density and helps compare houses of different sizes. `IsNewHouse` identifies recently built houses, which may have higher market value.

These features improve predictive power by converting raw columns into more meaningful information.

4. Encoding Decisions

Categorical variables were encoded based on their meaning and cardinality. Quality-related columns such as `ExterQual`, `KitchenQual`, `BsmtQual`, `GarageQual`, and `FireplaceQu` were label encoded because their categories have a natural order from poor to excellent. Nominal categorical variables with ten or fewer categories were one-hot encoded because they do not have a meaningful order and one-hot encoding prevents false ordinal relationships. High-cardinality variables such as `Neighborhood` were frequency encoded to avoid creating too many dummy columns. Columns with very high missing values or one dominant category were dropped because they could add noise rather than useful information. This encoding strategy keeps the dataset machine learning-ready while preserving useful category information.

5. Scaling Analysis

Three scaling methods were compared: StandardScaler, MinMaxScaler, and RobustScaler. StandardScaler transforms features to have mean 0 and standard deviation 1, which is useful for linear regression and other models sensitive to scale. MinMaxScaler transforms features into a fixed range between 0 and 1, but it is sensitive to outliers. RobustScaler uses median and IQR, making it better for datasets with strong outliers. For Week 4 linear regression, StandardScaler is the best choice because it keeps features centered and comparable, improving model training stability. Tree-based models usually do not require scaling because they split data using thresholds.

6. Skewness Treatment Findings

Skewness analysis showed that several numerical features were highly skewed. Features with absolute skewness greater than 0.75 were treated using `np.log1p()` when values were non-negative. For `SalePrice`, log, square-root, and Box-Cox transformations were compared. The log transformation worked well because house prices are naturally right-skewed and log transformation reduces the effect of extreme high prices. Skewness treatment is important because highly skewed variables can negatively affect linear models and make relationships harder to learn. Transforming skewed features helps create more balanced distributions and improves model readiness.

7. Reflection

The hardest concept in this week was choosing the correct preprocessing technique for each type of feature. Encoding, scaling, skewness treatment, and feature selection all require different decisions based on data type and distribution. The most surprising pattern was how strongly overall quality and total living area influence sale price. Another important learning point was that feature engineering can create stronger predictors than many raw columns. If I continued this project, I would train regression models such as Linear Regression, Random Forest, and XGBoost, compare their performance, and submit predictions to Kaggle. This week improved my ability to prepare professional ML-ready datasets.